진솔.

풀스택 개발자 · GCP 인프라 · 데이터



GCP 위에 사내 시스템 5종을 직접 세우고 운영합니다.

법무법인의 사내 개발을 맡아 인프라 설계부터 데이터 이관, 프런트·백엔드 구현, 장애 대응까지 한 사이클을 담당합니다. 구글시트로 돌아가던 전국 10개 지사 업무는 CRM으로, ASP.NET 레거시 홈페이지는 Next.js로 대체했고, Vertex AI 기반 RAG 상담봇을 직원 대상으로 시범 운영하고 있습니다. 5종 중 4종이 단독 구축입니다.

5개 제품

구축·운영 중인 사내 시스템

4개 단독 · 사건관리 앱은 2인 팀

API 423개

직접 설계한 엔드포인트

DB 85테이블 · 코드 23만 라인 · 커밋 1,873건

GCP 전담

인프라·DB·배포 단독 운영

Cloud Run · Cloud SQL · GCS · Cloud Build · Vertex AI

이 포트폴리오에서 보여드릴 역량

01 인프라를 직접 세우고 운영합니다

Cloud Run·Cloud Build·Cloud SQL·GCS·Secret Manager로 배포 경로를 구성하고, 도메인 전환과 장애 대응·사후 분석까지 맡습니다.

02 데이터 구조와 이관을 책임집니다

레거시 SQL Server 730만 건과 구글시트 9.8만 건을 PostgreSQL로 옮기고, 85개 테이블의 스키마 설계와 쿼리 튜닝을 직접 합니다.

03 제품을 화면부터 API까지 만듭니다

React·Next.js·Flutter 화면을 FastAPI·NestJS·Express에 연결하고, 실시간 갱신·구글 워크스페이스 연동·RAG 검색을 구현합니다.

주요 기술

Infra

GCP Cloud Run · Cloud Build · Cloud SQL · GCS · Secret Manager

Data

PostgreSQL · Prisma · SQLAlchemy · postgres_fdw · 대규모 이관

Backend

FastAPI(Python) · NestJS · Express(Node.js) · Vertex AI(RAG)

Frontend

React · TypeScript · Next.js · Flutter · TanStack Query/Table

01–03 포트폴리오 소개 04부터 경력기술서

사내 시스템 5종 중 대표 세 가지.

나머지 둘은 사내 포털과 AI 상담봇이며 상세 경력에 정리했습니다.

01 사건관리 앱

2인 팀 · 2026.06 – 현재

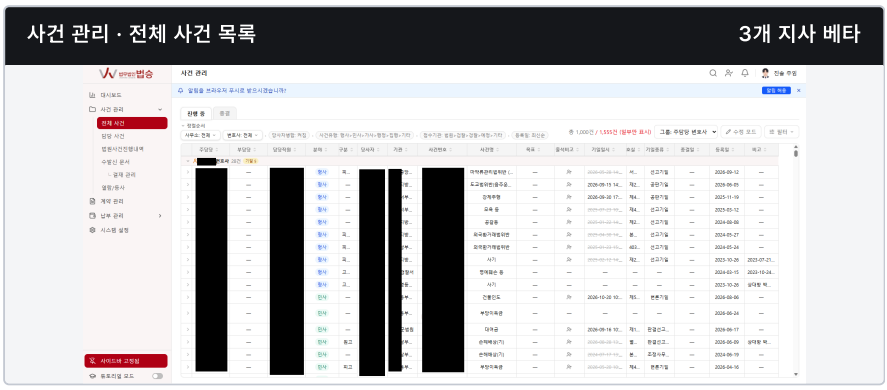
React · TypeScript · TanStack Query/Table · FastAPI · 3개 지사 베타

사무소·변호사·사건유형별로 필터링한 사건 목록에서 기일과 담당자를 한 화면에서 확인합니다.

API 189개·38테이블 규모의 사내 업무 도구를 2인 팀으로 개발하며 상담·문서·전자결재·정산 도메인과 배포를 맡았습니다.

담당 상담·문서·전자결재·정산 · 배포

→ 상세 경력



02 홈페이지 V2

1인 풀스택 · 2026.05 – 현재

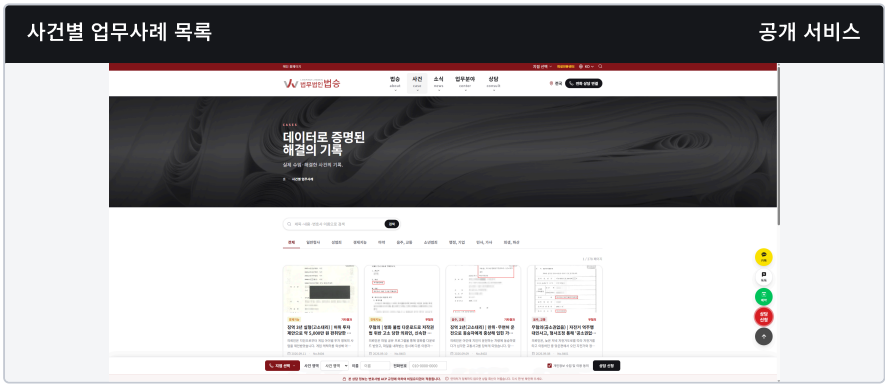
Next.js · NestJS · Prisma · PostgreSQL · 공개 서비스

공개 사이트에서 사건 유형·키워드로 해결 사례를 찾아보는 목록 화면입니다.

레거시 ASP.NET 홈페이지를 Next.js로 리뉴얼해 공개 38면·어드민 CMS 37면·API 146개를 1인으로 구축했습니다.

담당 공개 UI · 어드민 CMS · API · 인프라

↗ 서비스 보기 → 상세 경력



03 사내 CRM

1인 풀스택 · 2026.01 – 현재

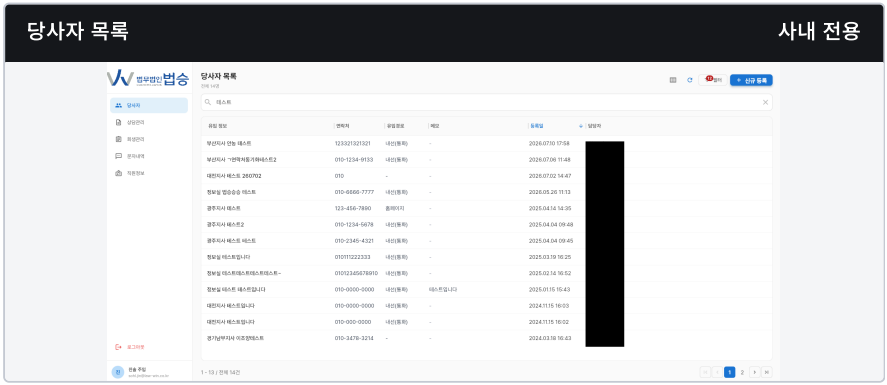
Flutter · Express · PostgreSQL · 사내 전용

유입 정보와 담당자를 연결해 상담 배정을 지원하는 당사자 목록입니다. (화면은 테스트 데이터)

전국 10개 지사 업무를 1인으로 구축했습니다. Flutter 54면·API 66개·17테이블, 구글시트 9.8만 건 이관.

담당 Flutter UI · API · DB 설계 · GWS 연동

→ 상세 경력



화면은 실제 서비스 캡처이며, 의뢰인 정보는 테스트 데이터로 대체하거나 가렸습니다.

화면·상태·데이터를 함께 설계합니다.

공통 UI와 상태 공유, Next.js 첫 화면, 업무 데이터 조회의 문제를 해결했습니다.

01 / React · 상태관리

4종 공통화

업무 목록의 공통 UI와 표·모바일 상태 공유

사건관리 · TanStack Table · TypeScript

문제 목록마다 표 설정이 달랐고 모바일 카드와 정렬 상태가 분리돼 있었습니다.

판단 공통 DataTable과 화면별 설정 저장을 도입하고, 표·카드의 정렬 상태를 페이지에서 공유했습니다.

결과 **4종 표를 공통화하고 6종 목록에 모바일 카드를 적용해 필터·정렬·합계를 유지했습니다.**

경력기술서에서 구현 상세 →

02 / Next.js · 렌더링

5.54MB → 589KB

첫 화면의 이미지와 콘텐츠 표시 지연 개선

홈페이지 V2 · 대표 이미지 원본 용량

문제 히어로 이미지 지연 로딩과 JS 실행을 기다리는 연출이 첫 화면 표시를 늦췄습니다.

판단 현재 이미지만 우선 로딩하고 WebP로 변환했습니다. 숨겨진 콘텐츠의 연출을 CSS로 옮겼습니다.

결과 **대표 이미지 원본 용량을 줄이고, JS 초기화를 기다리지 않고 본문을 표시하도록 개선했습니다.**

경력기술서에서 구현 상세 →

03 / Full Stack · 데이터

8초 → 0.05초

화면의 8초 대기를 데이터 조회 구조에서 해결

사건관리 · 상담료 처리 대기 조회

문제 CRM 원격 테이블과 로컬 테이블의 인라인 JOIN으로 목록 조회가 지연됐습니다.

판단 사건·스케줄을 먼저 조회한 뒤 이름·사무소 정보를 배치 조회하도록 변경했습니다.

결과 **해당 조회의 응답시간을 약 160배 개선하고 상담 확인·등록 업무를 한 화면으로 연결했습니다.**

경력기술서에서 구현 상세 →

PART 02 · CAREER EXPERIENCE

경력기술서.

근무 이력과 프로젝트별 담당 업무 · 기술적 의사결정 · 성과

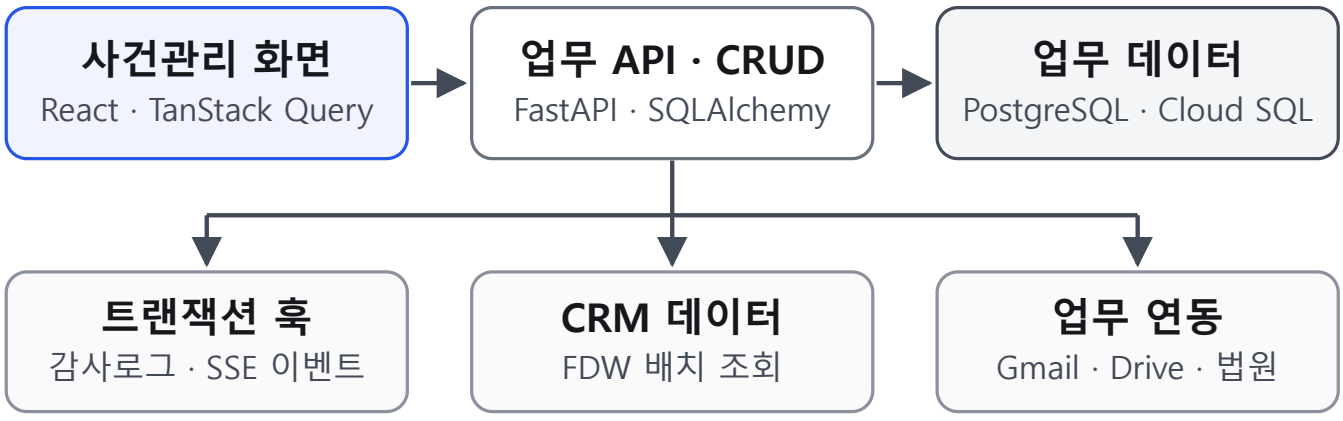
I. 사건관리 | II. 홈페이지 V2 | III. CRM
IV. 사내 포털 | V. 레거시 | VI. 인프라
VII–IX. 이전 경력

법무법인 법승 (2026-01-19 – 현재)

I. 사내 사건관리 앱 — 2인 팀 개발

- **기간**: 2026-06 착수 – 진행 중 (베타 운영: 일부 지사)
- **기술스택**: React + TypeScript + Vite(TanStack Query, TanStack Table, shadcn/ui) / FastAPI(Python) + SQLAlchemy + Pydantic / PostgreSQL(Cloud SQL, `postgres_fdw`) / APScheduler / Google Gmail·Drive·Workspace API / GCP Cloud Run
- **시스템 개요**: 상담료 · 수발신 문서대장 · 전자결재 · 열람등사 · 계약/미수금 등 변호사 실무 전 과정을 통합 관리하는 사내 업무 도구. **2인 팀 개발**이며, 본인은 **상담 · 문서 · 전자결재 · 정산(계약/미수금) 도메인**과 **인프라/배포 전반**을 담당. CRM과 데이터를 연동해 사건 · 당사자 정보를 단일 원천으로 관리.

아키텍처



핵심 흐름: React 화면 → API 라우터 → CRUD → PostgreSQL. 변경사항은 SQLAlchemy 세션 훅에서 감사로그와 SSE 이벤트로 연결하고, 브라우저에서는 관련 쿼리 캐시를 갱신합니다. CRM 원격 데이터는 인라인 조인 대신 배치 조회합니다.

담당 업무 및 핵심 문제 해결 사례

프론트엔드 A) 공통 DataTable·모바일 카드와 상태 공유

- **문제와 설계**: 계약·미수금·상담료·열람등사 목록의 반복 구현을 TanStack Table 기반 공통 DataTable로 통합. `useTableColumnState`에서 컬럼 폭·순서·표시·정렬을 관리하고 `tableId`별 localStorage에 저장해 화면마다 사용자

설정을 유지.

- **반응형**: 공통 모바일 카드 셀을 목록 6종에 적용해 필터·페이지네이션·합계를 유지. 정렬 상태를 표 내부에서 페이지로 올려 데스크톱 표와 모바일 카드가 동일한 값·비교 함수를 공유하도록 개선.

프론트엔드 B) 결재 업무 UX·대시보드 화면 디자인

- **업무 동선**: 결재 대기·완료·기안함 3탭과 `?tab=`을 연결해 새로고침·뒤로가기에도 선택을 유지. 알림에서 대기 목록 위로 문서 상세를 열어, 승인·반려 후 남은 업무를 이어 처리하도록 설계. 즉시 승인과 반려 사유 입력이 필요한 동작을 구분.
- **시각적 위계**: 대시보드를 월간 장부형 머리글, 경영 지표·업무 영역으로 재구성하고 색상 팔레트·KPI 타이포·위젯 테두리를 페이지 차원에서 통일.

프론트엔드 C) SSE·쿼리 캐시와 React 렌더링 안정성

- **서버 상태**: `fetch` · `ReadableStream` 구독으로 JWT를 인증 헤더에 전달하고 백오프·지터·탭 숨김 처리를 구현. SSE 변경 신호를 리소스별 queryKey 접두사와 연결하고 300ms 단위로 병합해 관련 목록·상세를 갱신.
- **렌더링 오류 해결**: 매 렌더마다 생성되는 기본 빈 배열의 참조 변경과 초기 데이터 미수신 처리를 분석. raw 응답과 렌더용 파생 배열을 분리하고 `undefined`와 빈 결과를 구분해 반복 렌더·첫 로드 전체 행 강조 오류를 해결.

1) FDW 원격 조인 병목 제거 — 응답시간 8초 → 0.05초 (약 160배)

- **문제**: 상담료 '처리 대기' 목록 조회에서, CRM 데이터를 `postgres_fdw`로 노출한 `crm.*` 스키마와 로컬 테이블을 인라인 JOIN하는 쿼리가 8초 이상 소요.
- **원인 분석**: FDW 포린테이블은 원격 조인이 로컬 옵티마이저에 푸시다운되지 않아, JOIN이 걸릴 때마다 원격 테이블 전체를 끌어와 로컬에서 재필터링하는 구조였음.
- **해결**: "스케줄(사건) 선조회 → 이름/사무소 배치(batch) 조회" 구조로 재설계해 원격 인라인 조인 자체를 제거.
- **성과**: 응답시간 8초 → 0.05초 (약 160배 개선).

2) 타입 불일치로 무효화된 인덱스 복구 — 1695ms → 109ms (약 15배)

- **문제**: 사건목록 조회가 매 요청마다 약 4만 행 Seq Scan을 반복하며 1.7초 가까이 소요.
- **원인 분석**: `party_detail_local.party_key` (`text`)와 `cases` / `matters.party_key` (`character(8)`, bpchar) 간 타입 불일치로 PostgreSQL 옵티마이저가 기존 인덱스를 사용하지 못하고 있었음.
- **해결**: 조인 조건에 `::text` 명시적 캐스팅을 적용해 인덱스 사용 경로를 복원.
- **성과**: 1695ms → 109ms (약 15배). 기일 충돌 판정 기준도 `< 3600` → `<= 3600`(60분 경계 포함)으로 함께 정교화.

3) SQLAlchemy 세션 훅 기반 "무수정(non-invasive)" 감사로그 + SSE 실시간 동기화

- **문제**: 20초 폴링 기반 화면 갱신의 실시간성 부족. 엔드포인트가 늘어날 때마다 감사로그 · 실시간 알림 발행 코드를 수동으로 삽입해야 하는 구조라 신규 엔드포인트에서 누락 위험 상존.
- **해결**: SQLAlchemy `before_flush` / `after_flush` / `after_commit` 세션 훅으로 ORM 변경사항을 전역에서 자동 포착 — `before_flush`에서 변경 diff를 수집, `after_flush`에서 PK 확정 후 **같은 트랜잭션**에 기록해 롤백 시 로그도 함께 사라지는 원자성을 보장. 동일한 훅 구조를 실시간 이벤트 발행(`core/events.py`)에도 재사용해 엔드포인트 50곳을 일일이 수정하지 않고도 전역 커버.
- **실시간 브로커**: Redis/Pub-Sub 대신 이미 쓰던 Cloud SQL의 **Postgres** `LISTEN/NOTIFY`를 채택(추가 인프라 비용 0). 전용 raw 커넥션을 별도 스레드에서 `select()`로 감시하고 `loop.call_soon_threadsafe`로 접속자별 asyncio 큐에 전달해, 동기 스택(psycopg2) 위에서 비동기 SSE 스트림을 구현. 프론트는 `EventSource` 대신 `fetch` + `ReadableStream`을 채택해 JWT가 쿼리스트링에 노출되지 않도록 처리.
- **성과**: 폴링 → 이벤트 기반 실시간 전환. 운영 실측(24시간) SSE 연결 620건 중 조기종료 0건.

- **운영 전용 장애 3건(로컬 재현 불가)**: ① `GZipMiddleware`가 최소 크기(1000B) 충족을 기다리며 SSE 신호를 무기한 대기하는 문제 → 경로 기반 판정으로 gzip 예외 처리. ② `is_disconnected()` 오판 → 애플리케이션 로그 대조로 실제 원인 규명 후 불필요한 체크 제거. ③ Firebase Hosting(Fastly CDN) 경유 시 스트리밍 버퍼링 → SSE만 Cloud Run에 직접 연결.

실시간 변경 전파 — 4단계

단계	처리와 설계 의도
1. 변경 수집	API 수정 요청 → 세션 훅에서 diff 수집 → 업무 데이터와 감사로그를 같은 트랜잭션에 저장
2. 커밋 후 발행	COMMIT 완료 후 PostgreSQL NOTIFY를 발행해 확정된 변경만 전달
3. 접속자별 전송	LISTEN 브로커 → 접속자별 asyncio 큐 → SSE 스트림
4. 화면 반영	브라우저가 이벤트를 수신해 관련 TanStack Query 캐시를 무효화하고 다시 조회

4) 전자결재(Approval) 시스템 — Gmail/Drive 위에 구축한 워크플로

- **구현과 업무 전환**: 초기 Gmail 링크·서명토큰 결재를 구현한 뒤, 7/28 메일은 알림, 승인·반려는 앱에서 처리하는 흐름으로 정리. 등록·결재 요청·안내메일 발송을 분리하고 발송 전 결재자 변경을 지원. 참조자(Cc)는 JSONB(`cc_approver_keys`) + GIN 인덱스와 기존 `approval_status`를 사용해 상태 이원화를 방지.
- **시도와 롤백**: Gmail 안에서 승인·반려하는 **AMP for Email**을 구현했으나, 발신자 등록에 필요한 **DKIM·DMARC 미설정**을 확인해 관련 코드를 롤백. 이후 메일 알림과 인앱 결재로 업무 동선을 통합.
- **재설계**: 결재 흔적을 파일명 prepend 방식에서 **Google Drive 라벨 배지(대기/완료/반려) 자동 동기화**로 재설계해, Drive에서 파일을 열지 않고도 결재 상태를 바로 확인 가능하도록 개선.

5) 열람등사(Record Request) 자동화 — '도과'의 재정의와 무인 알림

- **문제**: 열람 · 등사 신청 후 "도과"(기한 경과) 여부를 사람이 직접 계산 · 추적해야 했음.
- **해결**: '도과'를 "등사일자 미등록 상태로 방치"로 재정의해, 발의일 + 7영업일 도달 시 자동 알림을 발송(등록/종결 시 알림 중단)하도록 설계. 공공데이터포털 **특일정보 API**로 공휴일을 판정해 영업일 계산에 반영하고, 외부 스케줄러 없이 **인앱 APScheduler(매일 10:00 KST cron)** 로 자동화, 알림 채널을 카카오톡 웹훅 단일 경로로 통합.
- **트러블슈팅**: 이 API 키가 배포 과정에서 두 번 유실된 사고를 겪음 — 키가 없어도 앱이 죽지 않고 빈 공휴일 목록으로 조용히 폴백하는 구조라 증상이 바로 드러나지 않았고(캐시된 연도 데이터로 당장은 정상 작동, 연도가 바뀌면 조용히 틀어지는 시한폭탄 구조), 묶음 `.env` 대신 전용 Secret Manager 시크릿으로 분리해 "빠뜨리면 배포 자체가 실패"하는 구조로 재발을 방지.

6) 상담료 처리 대기 — 업무 흐름과 입력 상태 설계

- 목록과 처리 대기를 좌우 분할·독립 스크롤로 구성하고, 대기 항목 선택 시 의뢰인·상담일·유형·메모·상담 변호사를 입력 폼에 자동 반영. 예약·접수·완료·취소·노쇼 필터와 처리 동선을 구분해 조회부터 등록까지 연결. 7/1 .

7) 요청 단위 자동 커밋 구조에서의 동시성 · 일관성 제어 (실제 메커니즘)

- **설계·검토 이력**: 요청 단위 커밋 이후의 실행취소를 위해 역동작 클로저를 쌓는 Undo를 7/28에 구현·검토. 적용 범위·복구 내용 표시·중복 실행 차단까지 다뤘으며, 7/31 기능 폐기 결정으로 종료.
- raw SQL 변경은 세션 훅이 감지하지 못하는 사각지대라 `record_manual()` 헬퍼로 같은 세션 · 같은 트랜잭션에 수동 기록해, "롤백돼도 로그만 남는" 유령 기록을 방지.
- **트러블슈팅**: 감사로그 대상을 6개 → 22개 테이블로 확대한 직후, `executemany` 배치 삽입에서 insert/update가 혼합돼 행마다 파라미터 키 집합이 달라지며 배치 자체가 실패해 요청 트랜잭션 전체가 롤백되는 장애 발생. 액션과 무관하게 항상

동일한 키 집합을 채우도록 수정해 해결.

8) Cloud Run 빌드·배포 절차 표준화 — 명시적 이미지 배포와 응답 검증

- **개선 배경**: 소스 기반 배포에서 의도한 루트 Dockerfile 대신 프론트엔드 정적서빙 이미지가 배포되어, API가 HTML을 반환하는 문제를 분석하고 이전 리비전으로 롤백. 당시 영향은 각각 27분·9분의 서비스 중단으로 확인.
- **조치와 가치**: 루트 Dockerfile 명시 → 이미지 빌드 → `--image` 배포 → 응답 본문·Content-Type 검증의 절차를 정립. 배포 성공 여부와 실제 API 응답의 정상 여부를 함께 확인하도록 만들고, 이후 신규 시스템에도 같은 절차를 적용.

9) Cloud Run 실시간 서비스 설정 튜닝 (SSE 지원) + 프론트 · 백엔드 단일 서비스 통합 배포

- `--timeout 3600` (기본 300초로는 SSE가 5분마다 강제 종료), `--concurrency 250`, `--cpu 1000m` (1 vCPU 미만이면 최대 동시성이 1로 강제돼 SSE와 양립 불가) 등 제약을 파악한 뒤 적용.
- 프론트(Vite 빌드)와 백엔드(FastAPI)를 **단일 Cloud Run 서비스**로 통합 배포(멀티스테이지 Docker), `--min-instances`를 서비스 레벨로 관리해 예열창(평일 08:30–19:30 KST) 토글이 리비전 재배포를 유발하지 않도록 설계.

10) 자체 마이그레이션 프레임워크 및 스키마-모델 정합성 자동 검사

- Alembic 대신 순번 기반 자체 마이그레이션 러너 운용. 계약일 기준 재설정 마이그레이션을 운영 DB에 무중단 적용, 회계 컬럼 재구조화(`consultation_fees`를 `fee_kind` + `fee_group_id`로 분리) 시 "구코드와 신코드가 동시에 적재되면 금액이 2배로 집계되는" 위험을 근거로 배포 순서를 명시적으로 고정.
- 계약 등록 API가 "마이그레이션은 반영됐는데 SQLAlchemy 모델 컬럼만 누락되어" 전면 500 장애가 발생한 사고를 계기로, 쓰기 스키마 필드가 모델 컬럼의 부분집합인지 자동 검사하는 정합성 체크 도구를 도입.

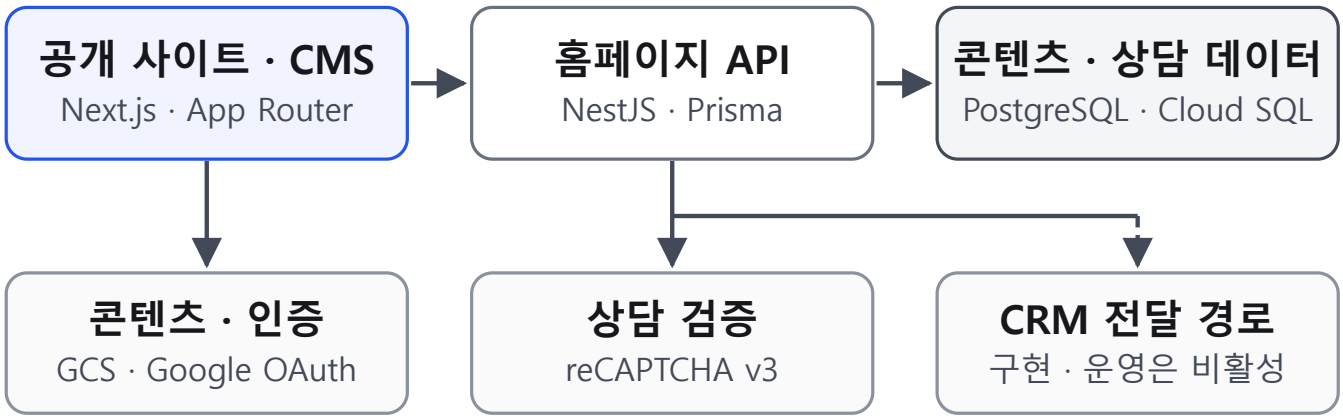
11) 회계 데이터 정합성 감사 (읽기 전용 전수 대사)

- 구글시트-앱 간 미수금 동기화가 **DB→시트 단방향**인 구조적 결함을 실측으로 규명. 월별 정산 차액을 두 가지 원인으로 분해해 금액 정합성을 검증.
- 조사 범위를 전 기간으로 확장해 DB 미수금 중복 데이터을 발견 — 전 과정 SELECT-only(운영 데이터 변경 0건)로 수행.

II. 법무법인 홈페이지 V2 리뉴얼 — 1인 풀스택

- **기간**: 2026-05 착수 – 진행 중
- **기술스택**: Next.js(App Router) + NestJS, TypeScript, Prisma, PostgreSQL(Cloud SQL, pgvector · pg_trgm, 레거시는 SQL Server), Turborepo, Tailwind, NextAuth/Google OAuth, GCP Cloud Run · Cloud Build · GCS · Cloud Scheduler, Vertex AI(Gemini 2.5 Flash · gemini-embedding-001)
- **시스템 개요**: 레거시 ASP.NET/SQL Server 기반 대고객 홈페이지(승소사례 · 변호사소개 등 홍보용 공개 사이트)를 Next.js 기반으로 전면 리뉴얼. 공개 사이트부터 사내 운영용 어드민 CMS까지 **스크래치부터 1인 풀스택**으로 구축.

아키텍처



상담 접수: reCAPTCHA 검증과 카테고리 조회를 병렬 처리하고, CRM 전달은 응답을 기다리지 않는 비동기 경로로 구현했습니다. GCS 업로드와 Google OAuth는 화면에서 사용하는 연동입니다.

연동 운영: CRM 전달은 8/7에 비활성화했으며, 8/24 보고에서도 `CRM_FORWARD_ENABLED=false` 상태를 확인했습니다. 10/8에는 광고 랜딩 전환 논의 중 "V2 상담은 CRM으로 넘기지 않고 구글챗 알림과 어드민 목록으로 받는다"로 확정했습니다. 도식의 점선은 구현된 전달 경로를 뜻합니다.

배포 경로: 사용자 → Global ALB → Cloud Run. 인증서는 Certificate Manager로 관리하며, 콘텐츠와 상담 데이터는 SQL Server에서 이관한 PostgreSQL에 저장합니다.

담당 업무 및 핵심 문제 해결 사례

프론트엔드 A) Next.js 서버·클라이언트 경계와 첫 화면 표시

- 서버 `SiteHeader`에서 전화 인터랙션을 클라이언트 `HeaderCallButton`으로 분리. 서버 HTML에 투명하게 내려와 JS 실행을 기다리던 등장 연출은 CSS로 옮겨 콘텐츠가 먼저 표시되도록 개선하고 `prefers-reduced-motion` 설정에도 대응.

프론트엔드 B) 이미지 로딩·공통 화면 디자인

- 현재 히어로 이미지에만 우선 로딩을 적용하고 다음 장면은 지연 로딩 유지. PNG 원본을 WebP로 사전 변환해 대표 이미지 용량을 **5.54MB → 589KB**로 축소.
- 전문센터의 히어로·본문·브랜드 스타일을 컴포넌트와 콘텐츠 데이터로 공통화.

프론트엔드 C) 반응형 어드민 UI — 표·카드 전환 기준 직접 수립

- **문제**: 어드민 목록 12개가 모두 `<table>` + `overflow-x-auto` 구조라, 열이 최대 9개인 화면은 좁은 폭에서 스크롤해도 내용을 읽을 수 없었음. 기존에 정해진 모바일 대응 기준이 없어(grep 0건) 화면마다 제각각 대응하던 상태.

- **해결**: **md(768px) 미만은 카드, 이상은 표**로 전환 기준을 직접 세우고 지사·약관·계정·변호사·센터·문의·게시물 등 12개 화면에 순차 적용. 정렬 화살표·체크박스 터치 영역을 22px·16px에서 36px·44px로 확대하고 삭제 동작은 `order-last`로 모바일 하단에 분리. 태블릿 구간(640~1024px)에서 저장 바가 셀보다 좌우 8px씩 밀려나던 여백 불일치도 마진 규칙을 맞춰 수정.

1) SQL Server → PostgreSQL 730만 건 데이터 이관

- Turborepo 기반 프론트/백엔드 모노레포 뼈대 구축과 함께, 레거시 SQL Server 데이터 **약 730만 건**을 PostgreSQL(Cloud SQL)로 이관 완료.

2) 상담 접수 API — `Promise.all` 병렬 처리 + fire-and-forget — 콜드스타트 대기 4초 제거

- **문제**: 상담 접수 시 reCAPTCHA 검증(구글 API 왕복)과 카테고리 조회(DB)가 순차 실행되어 응답 지연, 콜드스타트 시 최대 4초 대기.
- **해결**: `Promise.all`로 두 작업을 병렬화해 대기시간을 겹치고(`backend/consult.service.ts`), CRM 요청함 전달은 결과를 기다리지 않는 **fire-and-forget**으로 전환 — CRM 측 장애가 홈페이지 상담 접수 자체를 막지 않도록 장애를 격리.
- **성과**: 콜드스타트 최대 4초 대기 제거.

3) SEO/AEO — 검색 · 답변엔진(AI) 양쪽에 최적화

- **레거시 주소 보존 이전**: 옛 주소(`/{지역}/wincasedetail`)를 지역 · ID 그대로 보존한 채 V2 경로로 301 이전, 페이지 자기 canonical 위임 처리(홈으로 canonical이 흡수되던 버그 해소).
- **사이트맵 정리**: 2024년 옛 주소 **2,328건**이 담긴 정적 sitemap을 제거하고 동적 sitemap으로 통합.
- **답변엔진 엔티티 혼동 대응**: Perplexity · Gemini · 네이버 CUE 등 AI 답변엔진이 법인을 다른 개체와 혼동하는 문제에 대응해, `LegalService` JSON-LD에 공식 SNS와 지사 10곳의 네이버 플레이스 `sameAs`를 주입. 칼럼 · 사례의 저자를 담당 변호사(`Person`)로 귀속해 E-E-A-T(전문성 · 권위 · 신뢰성) 신호를 강화.

4) 보안 — 상담자 개인정보 · 어드민 접근 통제

- 권한 없는 어드민 계정에는 상담 내용 **PII 마스킹** 처리. 전역 보안 헤더(클릭재킹 · MIME 스니핑 방어) 적용, 상담 접수 API에 **IP 기준 빈도 제한**. **reCAPTCHA v3** 봇 차단 로직을 구글 장애 시 fail-open(통과)이 아니라 **fail-closed(차단)** 로 전환해 봇 유입 리스크를 줄임. 어드민 로그인을 Google 전용으로 일원화.

5) CMS — 어드민 콘텐츠 관리 기능

- 예약 발행(발행 시각 도래 전 404 차단), 이미지 드래그 업로드(로컬 → GCS 직접 업로드), 자동 목차(TOC) 생성, 20MB PDF 첨부, 최대 1만 건 CSV 다운로드(한글 인코딩 깨짐 방지 + 개인정보 마스킹). Google Workspace **역할 기반 인증 + 하이브리드 폴백**으로 어드민 계정 관리를 일원화. 다만 이 연동은 운영에서 한 번도 동작하지 않았고 폴백이 그 사실을 가리고 있었다 — 9/18에 찾아 고친 경위는 아래 16).

6) Cloud SQL 보안 7항목 전수 점검 및 리스크 판단

- 삭제 보호 비활성(백업 7일치와 함께 영구 삭제될 수 있는 상태) 즉시 조치, 평문 접속 허용 구간을 `ENCRYPTED_ONLY`로 전환(Cloud Run이 커넥터 소켓 경유임을 사전 확인해 운영 무중단 보장).
- 반대로, **멀티리전 복제는 의도적으로 미실행** — 한국 GCP 리전이 서울 하나뿐이라 복제본이 일본에 놓여 상담자 개인정보 국외이전이 되는 리스크를 근거로 보류. "경고를 없애는 조치"와 "리스크를 새로 만드는 조치"를 구분해 판단.
- **부작용 대응**: SSL 강제 조치 직후 마이그레이션 도구 12개가 인증서 검증 방식 차이로 파손된 것을 실측 확인 후 libpq 호환 플래그로 수정.

7) Cloud SQL 커넥터 순간 장애에 대한 앱 레벨 내결함성

- **문제**: Cloud SQL 커넥터가 순간 타임아웃될 때 `PrismaService.onModuleInit`이 재시도 없이 예외를 던져 Nest 부팅 자체가 실패 → Cloud Run이 인스턴스를 `exit(1)`로 종료 → STARTUP probe 실패로 6분 39초 전면 500 장애.
- **해결**: 기동 시 0.5초–8초 간격 5회 재시도, 재시도 소진해도 예외를 던지지 않고 기동을 이어가 DB 복구 시 자동 재연결하도록 변경.

8) 배포/도메인 컷오버 인프라

- Cloud Run 도메인 매핑이 서울 리전을 지원하지 않음을 확인한 뒤, 글로벌 외부 로드밸런서(ALB, 고정 IP) + Certificate Manager(DNS Authorization 방식 인증서)로 경로 전환 — 다운타임 0으로 컷오버 완료.
- Kaniko → BuildKit 빌드 전환으로 프론트 빌드시간 3분 4초 → 2분 23초 단축. `.gcloudignore` 적용으로 배포 업로드 용량 **7.6GB → 115MB**로 축소.
- 레거시 GSC 미색인 17.8만 건 원인 분석 — 필터 조합 URL이 크롤 예산을 소진하는 구조적 문제를 확인해 `robots.ts`로 차단.

AI 상담봇(RAG) — 검색 평가부터 공개 판단까지

- **목적**: 방문자의 사연을 담당 변호사가 바로 전화할 수 있는 상담 접수 1건으로 다듬어 넘기고, 비슷한 사건의 실제 대응 사례를 근거로 소개한다. 결과 예측과 수임 권유는 하지 않는다.
- **구성**: 사이트 공개 글 → 청킹·임베딩(pgvector) → 의미 검색 + 글자 검색 순위 합산(RRF) → Gemini 생성(SSE 스트리밍) → 출력 검사 → 대화 저장. 백엔드 `chat` 모듈 22파일 · 약 3,400 LOC.
- **현재 상태**: 2026-09-23 공개 후 9/28 **직원 전용으로 회수**, 동의·파기 요건을 갖춘 뒤 직원 대상 시범 운영 중. 외부 재공개는 변호사 광고 규칙 판단 대기(아래 12).

9) 검색 층과 평가 체계 — 골든셋 50문항, Recall@5 95.8%

- **색인**: 사이트 글 4,398건을 300~800자로 나눠 **7,786조각** 적재. 내용이 그대로인 조각은 다시 임베딩하지 않아 재실행 비용을 낮추고, 9/29부터 Cloud Scheduler가 매주 바뀐 조각만 갱신.
- **채점 기준을 먼저 만듦**: 실제 상담 접수 본문에서 말투를 뽑은 24문항 + 분야를 메우는 24문항 + **아무 답도 하지 않아야 정답인 2문항**으로 골든셋 50문항을 구성(개인정보 전부 제거).
- **판정선 정정**: 질문 7개로 정한 유사도 판정선 0.28이 50문항에서는 정답·무관 구간이 겹쳐, 양쪽을 다 지키는 0.32로 정정 — 표본이 작은 수치를 근거로 쓰지 않는다는 기준으로 남김.
- **RRF**: 의미 검색은 사건번호·법조문에, 글자 검색은 동의어에 약해 두 순위를 합산 → Recall@5 93.8% → **95.8%**.
- **법률 문서에서 거꾸로 작동한 일반 기법**: 흔한 단어를 빼 잡음을 거르려 했으나 실제 빈도가 집행유예 9.85% · 보이스피싱 6.64%, 오늘 2.30% · 저녁 0.57%로 **법률 용어가 흔하고 일상어가 드물었다.** 단어 빈도 대신 "의미 검색이 아무것도 못 찾으면 글자 검색 결과도 쓰지 않는다"는 질문 단위 게이트로 교체.
- **내린 글 인용 차단**: 색인은 적재 시점 스냅샷이라, 공개 여부만은 매 질의마다 원본 테이블에서 확인.

10) 돌려보고서야 드러난 가드레일 구멍 — 출력 검사를 두 시점으로

- **문제**: 화면 없이 `질문 → 검색 → 답변`을 터미널에서 돌려 보니 ① 금지한 결과 표현("집행유예 성공")을 본문에서 재생산 ② **없는 대표번호와 오타 도메인을 생성** ③ 번호를 막자 없는 지사 목록을 나열. 프롬프트로 금지해도 다른 자리에서 새어 나왔다.
- **해결**: 전화·주소는 모델이 아니라 **코드가 검증된 지사 정보로 붙이고**, 사무소 목록은 사실로 주입. 스트리밍은 이미 보낸 글자를 회수할 수 없으므로 금지 표현은 조각이 아니라 **누적문으로 검사해 보내기 전에** 끊고, URL·연락처 허용 목록 검사는 잘린 주소를 오판하지 않도록 **응답 완료 후**로 분리. 실제로 새어 나왔던 문장을 회귀 테스트로 고정.

11) 운영에서만 보이는 실패 — 운영 DB 미적용 · 서울 리전 429

- 마이그레이션 SQL은 커밋됐지만 운영 DB에 적용되지 않아 첫 요청에서 테이블 부재가 드러남 → "DB → 소량 색인 → 전체 적재 → 운영 SSE 확인 → 프론트 배포" 순서를 문서로 고정.
- "문의가 몰려" 하고 상담으로 넘기던 답이 가드가 아니라 **서울 리전 공유 용량 부족(429)** 이었음을 로그로 확인. 서울·도쿄 비교 측정 후 **생성만 도쿄로 분리**하고 검색·저장은 서울에 유지. 대화 서비스가 실패를 그럴듯한 문장으로 덮지 않도록 "일시적인 오류 + 다시 보내기"로 정직하게 표시.
- 재시도는 횟수가 아니라 요청의 남은 시간 예산으로 제어하고, 스트리밍이 시작된 뒤에는 재시도하지 않음.

12) 공개 회수와 개인정보 · 광고 규정 요건

- **판단**: 9/23 공개 후 개인정보 처리방침과 파기 장치 없이 연휴 동안 실제 사연이 쌓인 것을 확인하고, 9/28 첫 작업으로 어드민 로그인 직원에게만 보이게 회수(API 직접 호출도 401). 공개 기간 대화는 결정 후 파기.
- **요건 구현**: 대화 전 수집·이용 · 민감정보 · 국외 이전 동의를 각각 받고 품질 개선은 선택 동의로 분리, 서버가 동의 판본을 검증. 동의 90일 / 미동의 24시간 보관 후 매시간 파기. 개정 처리방침이 게시되면 **서버가 읽어 동의 방식을 스스로 전환** — 사람이 스위치를 기억해 바꾸는 절차를 없앰.
- **광고 규정**: 변호사 광고 규칙 제5조②(소비자가 AI를 직접 사용하게 하는 광고 금지)가 유지되고 있어, 방문자 대상 공개는 광고책임변호사 판단을 공개 조건으로 추가.

13) 접수 경로 일원화 — 대화를 상담 신청서로 넘김

- 상담봇 안에 별도 접수 폼을 두면 제한·알림·전환 추적을 두 벌 관리해야 해서, 대화를 영역·제목·상담내용으로 요약해 **기존 상담 신청 폼에 채워 넘기도록** 변경. 사연은 URL이 아니라 `sessionStorage`로 넘겨 분석 도구로 새지 않게 하고, 요약이 틀려도 원문으로 확인할 수 있게 대화 원문을 함께 붙임.

2026-09 ~ 10 운영 고도화

14) 사례 3,038건에 AI 요약 · FAQ — 생성 · 자동 검사 · 공개 파이프라인

- **구조**: 공개 사례를 조회수 순으로 30건씩 Gemini에 넣어 요약 3줄과 FAQ 초안을 만들고, 검수 상태·모델·토큰은 레거시 동기화가 닿는 원본과 분리한 별도 테이블에 저장. 생성 5분 · 승인 10분 스케줄러로 새 사례도 자동 공개되며, FAQ를 못 읽어도 글은 원래대로 나간다.
- **자동 검사**: 끝난 사건을 설명하는 글이라 결과어(무혐의 등)는 허용하고, **원문에 없는 숫자** · 비율 · 보장 · 최고 · 전관 · 무료 상담 · 일반론을 잡아 비공개로 돌림. 측정값 퍼센트("혈중알코올농도 0.260%") 오탐은 비율 규칙에서만 예외 처리.
- **규정 해석이 설계를 바꿈**: 처음엔 검토 변호사를 지정해야만 공개되게 만들었으나, 규정 제6조가 "AI 이용 사실을 광고할 때"만 적용되고 상위 규칙 제5조①이 2025.6.30 삭제된 것을 확인해 **AI 표기 없이, 실제로 읽은 변호사만 표시**하는 형태로 변경.
- **성과**: 공개 사례 3,358건 중 **3,038건 공개**, 303건은 검사에 걸려 비공개, 실패 0. 운영 투입 당일 첫 배포에서 시험 스크립트가 빌드에 섞여 새 리비전이 뜨지 못했으나 직전 리비전이 계속 서비스해 영향 없음.

15) 바뀐 글을 검색엔진 · AI에 알리기 — IndexNow + 사이트맵 수정일

- **문제**: 레거시 스키마에 수정일 칸이 없어 FAQ를 붙인 3천 건도 크롤러에게는 "안 바뀐 글"이었고, 사례 본문을 고쳐도 FAQ에는 옛 결과가 남을 수 있었음.
- **해결**: 공개 글 · 변호사마다 내용 지문(md5)을 저장해 10분마다 비교하는 모듈 하나로 ① 빙 · 네이버 IndexNow 통지(빙 색인 = ChatGPT 검색) ② 사이트맵 `lastmod` · Article `dateModified`(구글 = Gemini) ③ 본문이 바뀐 사례의 FAQ 재생성을 함께 움직임. 마이그레이션을 먼저 하지 않으면 FAQ가 사라지는 순서 의존성을 체크리스트로 고정.
- **성과**: 첫 통지 7,227건(게시물 7,189 + 변호사 38), 사례 3,039건의 수정일이 FAQ 공개일로 갱신.

16) "조용한 실패" 3건 — 실패가 폴백에 덮여 화면이 정상으로 보이던 구조

- **레거시 동기화 2주 침묵**: 배치의 마지막 명령이 `echo`라 종료코드가 늘 0이어서 스케줄러에는 성공으로 떠 있었음. 실패 즉시 알림과 별도로 **26시간 침묵 감시**를 다른 기계에 둬, "실패"뿐 아니라 "아예 안 돈 것"도 잡히게 함(구조 전환과 복구 상세는 V절 10).
- **상담 번호 충돌**: 이관 이후 V2와 레거시가 같은 번호대에서 번호를 각자 발급해 42개 번호에 서로 다른 사람이 앉아 있었음 → 접수 고유키로 대조하고 넣을 때 새 번호를 발급해 **305건 복구**. 게시물은 건너뛰면 글 하나가 빠지지만 상담은 전화를 기다리는 사람이 사라진다.
- **구글 권한 연동 0%**: 맞춤정보 묶음 이름이 코드와 달라 **직원 109명 전원의 역할을 한 건도 읽지 못했고**, 못 읽으면 조용히 DB 권한으로 넘어가 몇 달간 드러나지 않았음. 이름을 바로잡고 어느 이름으로 찾았는지 로그에 남기며, 복수 권한 합집합 판정 · 화면 전환마다 재확인(조회 실패와 권한 없음 구분)으로 보강.

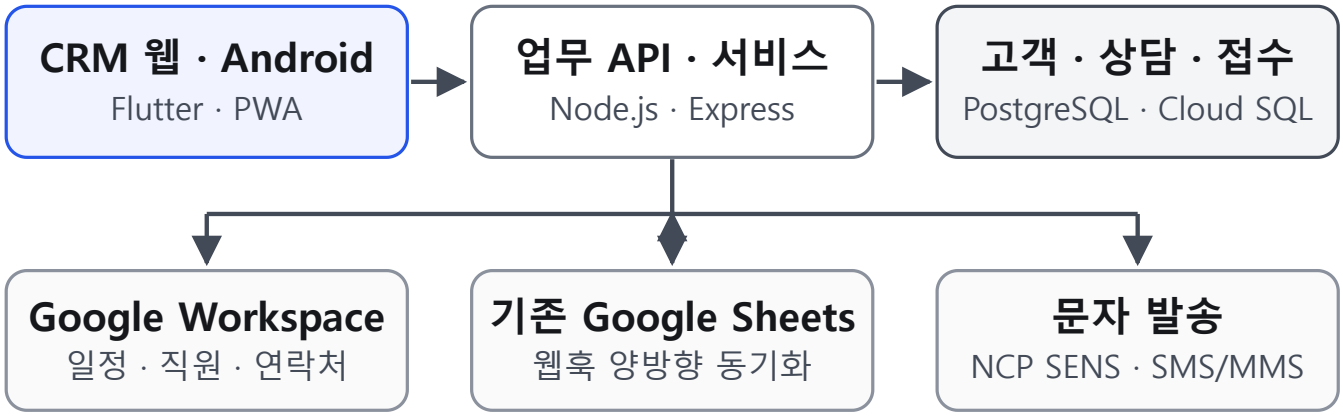
17) 측정을 살리고 숫자를 교차 확인

- **조회수 자체 집계**: V2에는 조회수를 올리는 코드가 없어 이관 때 값에 멈춰 있었음. 캐시된 상세 페이지에서는 브라우저가 진입을 알리고 백엔드가 판정(봇 · 10초 중복 제외, IP 위조 방지를 위해 프론트 서버만 호출 가능).
- **AI 유입 측정**: GA4 Data API로 두 홈페이지를 같은 잣대로 비교하고(AI 유입 세션의 약 80%가 레거시 도메인으로 감), 서버 로그로 교차 확인. AI 에이전트 이름을 사칭한 비밀 파일 탐색 스캐너(전부 404)를 집계에서 걸러냄. gcloud 기본 클라이언트로는 애널리틱스 범위가 막혀 전용 OAuth 클라이언트를 만들고 비밀값은 Secret Manager에 보관.

III. 사내 CRM — 1인 풀스택

- **기간**: 2026-01 착수 – 진행 중 (정식 오픈 2026-05)
- **기술스택**: Node.js(Express) 백엔드 + Flutter(Dart) 클라이언트(PWA + Android) / Google Sheets(Apps Script) → MySQL → PostgreSQL(Cloud SQL) / Google Workspace API 연동
- **시스템 개요**: 전국 10개 지사의 고객 유입 → 변호사 배정 → 상담 예약 → 문자 안내로 이어지는 전 과정을 통합 관리하는 맞춤형 CRM. **스크래치부터 1인 풀스택으로 구축**(API 54개, 16개 테이블, 약 49,000 LOC 규모).

아키텍처



데이터와 연동: 고객·상담·접수 정보를 PostgreSQL에서 관리합니다. 홈페이지 접수용 요청함 연동 경로를 CRM API에 구현했고, 사건관리는 CRM API 및 FDW로 관련 정보를 조회합니다. 데이터 저장소는 Google Sheets → MySQL → PostgreSQL 순서로 전환했습니다.

담당 업무 및 핵심 문제 해결 사례

1) Google Sheets(Apps Script) → Cloud SQL → PostgreSQL — CRM의 시작점을 직접 이관

- **배경**: CRM 구축 이전, 전 직원이 구글시트에 당사자 · 상담 · 문자 기록을 수기로 관리하고 있었음.
- **구현**: 1차로 당사자 · 상담 · 문자/전화 등 **약 9.8만 건**을 Cloud SQL로 이관하는 스크립트를 작성해 초기 적재. 이후 **Apps Script 트리거 → 웹훅 → DB 반영**의 실시간 양방향 동기화 구조를 자체 구축해, 신규 CRM과 기존 구글시트 워크플로가 과도기 동안 공존하도록 설계. 최종적으로 MySQL(Cloud SQL)을 거쳐 PostgreSQL(Cloud SQL)로 이관까지 전 과정을 단독 수행.

2) [성능] 서브쿼리 480개 제거 + 복합 인덱스 + `Promise.all` 병렬처리

- **문제**: 당사자 목록 조회가 행 단위 서브쿼리 구조라, 페이지당 최대 480개의 서브쿼리가 발생해 조회 속도 저하 및 무한 로딩 현상 발생.
- **해결**: 서브쿼리 구조를 `LEFT JOIN`으로 통합해 재작성. 상태(`status`)와 등록일(`created_at`)에 **복합 인덱스**를 추가해 목록 조회의 주요 필터 조건을 인덱스로 커버. 데이터 조회 쿼리와 카운트(전체 건수) 쿼리를 `Promise.all`로 병렬 처리해, 두 쿼리의 순차 대기 시간을 겹쳐 화면 대기 시간을 단축.

3) [동시성] Cloud Run 다중 인스턴스 스케줄러 중복 실행 — MySQL Named Lock(`GET_LOCK`)

- **문제**: Cloud Run이 다중 인스턴스로 확장될 때, 동일한 자동 동기화 스케줄러(Google Admin 동기화 등)가 인스턴스마다 중복 실행되어 데이터 중복 처리 위험 발생.

- **해결**: Redis 등 추가 인프라 도입 없이, **MySQL Named Lock(`GET_LOCK`)** 으로 스케줄러 실행을 인스턴스 간 상호 배제(mutual exclusion) 처리 — 하나의 인스턴스만 락을 획득해 실행하고 나머지는 스킵하도록 구현. 이는 SQL 세션에 종속된 어드바이저리 락으로, 별도 락 테이블이나 외부 캐시 없이 DB 커넥션만으로 분산 잠금을 구현한 방식.
- **함께 처리**: 직원 식별 기준을 이름에서 `employee_key` 로 통일하고, 레거시 데이터에는 폴백 로직을 적용해 서비스 중단 없이 식별체계를 전환.

4) 동시 접수 처리 충돌 방지 — Claim(점유) 기반 동시성 제어

- **문제**: 다수 직원이 동일 홈페이지 접수 건을 동시에 열람 · 처리하면서 데이터가 서로 덮어써지는 충돌 발생.
- **해결**: 접수 처리 시작 시점에 해당 건을 즉시 점유(claim) 상태로 전환하는 컬럼/로직을 추가하고, **10분간 미조작 시 자동 회수**되는 타임아웃을 설계 — 애플리케이션 레벨의 **낙관적 점유(claim) 방식**을 채택해, 다른 직원 화면에는 "처리 중" 잠금 표시로 즉시 충돌을 차단.

접수 점유 흐름

상황	처리
직원 A가 처리 시작	접수 건에 claimed_by·claimed_at을 기록하고 처리 화면 진입
직원 B가 같은 건 접근	점유 상태를 확인해 '처리 중'으로 표시하고 수정 제한
10분간 미조작	점유가 만료되어 다른 직원이 이어서 처리 가능

5) Google Workspace 연동 — Calendar / Admin Directory / People API

- 상담 등록 · 수정 · 삭제 시 Google Calendar 이벤트를 자동 반영(다중 상담사 지원), 지사별 고유 캘린더 매핑. Google Admin Directory · People API로 직원 계정 동기화, 고객 연락처 저장, 일정 생성을 자동화.
- 직원 식별체계를 이름 기반에서 `employee_key` (고유키)로 전면 통일 — 개명 · 표기 흔들림에도 안전하게 연결.

6) NCP SMS/MMS 발송 — 바이트 기준 타입 자동 판정 + 예약 · 템플릿 시스템

- 문자 길이를 글자 수가 아닌 **실제 바이트 수** 기준으로 SMS/LMS/MMS 타입을 자동 판정해 발송 오류를 방지. 예약 전송과 템플릿 시스템을 자체 구축. MMS 이미지 자동 변환(PNG→JPEG, 비율 유지 최적화)과 GCS 인증 폴백 처리를 구현.

7) 안정성 — DB 자동 재연결, Rate Limit, OAuth 화이트리스트

- MySQL/PostgreSQL 커넥션 끊김 시 자동 재연결 및 503 응답 변환 처리로 순간 장애가 전체 장애로 번지지 않도록 방어. 별도 라이브러리 없이 **직접 구현한 Rate Limit** 미들웨어로 로그인 · 문자 발송 등 민감 API에 호출 제한 적용. 도메인 화이트리스트(`@law-win.co.kr`) 기반 OAuth로 인가되지 않은 계정의 접근을 원천 차단.

8) Flutter 클라이언트 — PWA + Android 크로스플랫폼

- GoRouter 기반 라우팅, Google 네이티브 로그인(`google_sign_in`), Android 빌드환경(Java 17 + AGP 8.2.1) 대응. PDF/PNG 문서 출력(`printing` / `pdf` 패키지 + 한글 폰트 임베딩). 크로스플랫폼 빌드 형상관리: OS별 개행문자 차이로 발생하는 가짜 diff를 `.gitattributes` / `.gitignore` 로 제거.

회고 — Sheet 기반 레거시를 RDBMS로 전환하며 성능 · 동시성 · 보안을 1인이 전담했습니다. 4개월간 643개 커밋 중 SMS · 타임존 관련 수정이 가장 많았는데, UTC↔KST 변환과 외부 발송 API의 엣지 케이스를 다루며 "기능 구현보다 운영 환경의 디테일이 더 어렵다"는 점을 체득했습니다.

IV. 사내 업무 포털 — 1인 풀스택

- **기간**: 2026-08-31 착수 – 운영 중 (portal.lawwin.co.kr, 사내 전용)
- **기술스택**: Next.js 16(App Router, 서버 컴포넌트 + Route Handler) · TypeScript · Auth.js v5(Google, JWT 세션) · React Flow + dagre · Tailwind v4 / GCS(조직도 · 예약 저장) / Google Workspace API(Calendar · People · Admin Directory · Chat) / LG Centrex 클릭투콜 / GCP Cloud Run + Firebase Hosting
- **시스템 개요**: 조직도를 PDF 썸네일로 박아 두어 편집·검색·연동이 불가능하던 Google Sites 사내 포털을 대체. 오늘 일정 · 구성원 명부 · 조직도 · 내선 통화를 한 화면에 모으고, 크롬 익스텐션에 있던 구성원 계정 관리까지 흡수. **별도 백엔드와 DB 없이** Next.js 서버 하나가 화면과 API를 함께 맡는다. 페이지 5 · Route Handler 22 · 약 1.5만 LOC · 커밋 90건(단독).

아키텍처

요청 흐름: 브라우저 → Firebase Hosting(전 경로를 Cloud Run으로 리라이트) → Cloud Run `lawwin-mainboard` → GCS(조직도 · 퇴사 예약 · 로그인 쿠키 스냅샷) · CRM API(재직자 명단 · 권한 판정) · 구글 어드민(계정 · 회선 · 지사별 내선) · 수신 번호표 Job.

값마다 원본은 한 곳: 이름 · 내선 · 직위는 CRM(구글 어드민의 웹훅 사본), 배치 · 순서 · 배지는 포털의 조직도 파일, 계정과 지사별 내선은 구글 어드민. 두 데이터의 변경 주체와 빈도가 달라 한쪽에 몰면 어긋난다는 판단으로 나눴다.

담당 업무 및 핵심 문제 해결 사례

1) DB 없는 조직도 — 원본 PDF 좌표 추출과 CRM 일원화

- 조직도는 쓰기 빈도가 낮고 이력이 중요한 데이터라 테이블 대신 JSON 스냅샷으로 저장하고, 저장할 때마다 변경 내역과 함께 버전 파일을 쌓는다.
- 스크린샷 OCR 대신 배포본 PDF의 텍스트를 좌표째 추출해 88명을 옮겼고, 손으로 옮긴 1차 전사본의 오탈자 8건이 그 과정에서 드러났다.
- 이름 · 내선 · 직위를 CRM에서 끌어오도록 바꾸고 사원키로 연결해 개명·표기 차이에도 깨지지 않게 했다. 88명 × CRM 377명 대조에서 매칭 실패 0.
- 조직도 PDF가 공개 폴더에 있어 로그인 없이 내려받히던 대외비 노출 경로를 막았다.

2) 로그인 — 매번 뜨던 동의 화면과 Firebase Hosting 쿠키 제거

- "앱마다 로그인을 눌러야 한다"는 불편의 원인이 직접 넘기던 `prompt=consent`(세션과 무관하게 매번 동의 화면)였음을 실제 인증 URL을 뜯어 확인하고 제거. 같은 원인을 CRM · 사건관리 앱에도 확장 적용.
- Vercel → Cloud Run 이관(월 $20 → $0) 후 로그인만 500으로 실패. 세 번의 가설(바인딩 주소 · 절대 URL · 호스트 정규화)로도 증상이 남아, 운영 응답 쿠키를 재전송하는 **재현**으로 **Firebase Hosting이 `__session` 외 쿠키를 모두 지운다**는 것을 확정. 쿠키 묶음은 비공개 GCS 스냅샷에 두고 브라우저에는 256비트 임의 식별자만 내보내는 브리지로 해결.
- 로그인 허용을 이메일 끝자리 검사에서 ID 토큰의 워크스페이스 도메인(`hd`) + `email_verified` 검사로 강화 — 회사 주소로 만든 개인 구글 계정이 통과하던 구멍.

3) 내선 통화 — 누른 지사의 내선으로, 운영 403 해결

- 지사마다 전화기를 둔 변호사는 어느 지사 칸에서 눌러도 대표 내선이 울렸다. 구글 어드민에 지사별 내선을 적게 하고 누른 지사를 발신 요청에 실어 서버가 고르게 함. 못 읽으면 짐작해 걸지 않고 막는다.
- 운영에서 발신이 전부 403이던 원인은 Origin 검사가 Cloud Run 내부 주소(`0.0.0.0:8080`)와 비교한 것 — 공개 주소 기준으로 수정.
- 책상 전화기를 먼저 울리는 클릭투콜의 한계는 모바일 카드의 직장 · 휴대폰 `tel:` 버튼으로 풀었다.

4) 구성원 관리 화면 — 크롬 익스텐션 대체와 권한 fail-closed

- **권한 구멍**: 편집자 명단(`ORG_EDITORS`)이 운영에서 비어 있어, 빈 명단이 "검사 안 함"으로 읽혀 로그인한 전 직원이 조직도를 저장할 수 있었음. 명단을 폐지하고 "CRM 주 지사가 정보실인 재직자"로 **요청마다 판정**, CRM을 못 읽으면 아무도 못 고치게(fail-closed) 바꿈.
- **동시 저장**: 조직도 저장을 GCS 세대 번호 조건부 쓰기로 바꿔, 그 사이 누가 먼저 저장했으면 409로 거절하고 이력에도 남기지 않는다.
- **구조 결정**: CRM에 조직도 API를 두는 안은 "CRM이 쓰지 않는 값"이라 취소, 기존 관리 API 서버 수정안은 저장소 쓰기 권한이 없어 접고, **포털이 구글 어드민에 직접 쓰는 구조**로 하루에 배포까지 진행. 맞춤속성은 구글 스키마 정의를 읽어 맞추고, 없는 칸에 값을 주면 조용히 버리지 않고 400.
- **보호 장치**: 계정 추가(임시 비밀번호 1회 표시) · 수정 · 정지 · 퇴사 예약(퇴사일 다음 날 고용상태 퇴직, 정한 시각 계정 정지)의 모든 변경에 바뀌는 칸을 요약한 확인 창. 내 계정과 구글 관리자 계정의 비밀번호 재설정은 화면과 서버 둘 다 차단. 구성원을 고치면 수신 알림 번호표 Job을 바로 다시 돌려 최대 30분 옛 값으로 가던 수신 알림을 없앰.

5) 조직도를 보는 화면에서 편집하는 화면으로

- 정보실은 사람 칸을 누르면 사이드 패널에서 배치(역할 · 배지 · 순서)와 구성원 정보를 고치고, 박스 안에서 끌어 순서를 바꾼다. 박스를 누르면 CRM 조직 정보까지 그 자리에서 고치며, 확인 창에 다른 앱 영향(상담 일정 동기화, 사건 Drive 폴더)을 적는다. 변경은 초안으로 모았다가 확인 후 한 번에 저장.
- 퇴사자는 CRM 상태로 모든 박스에서 빼고, 저장한 값은 CRM 웹훅을 기다리지 않고 먼저 그린다. 일반 직원에게는 같은 칸이 연락처 카드(내선 통화 · 구글챗 · 이메일)로 열린다.

6) [성능] 로그인 직후 첫 화면 7.3초 → 1.5초 — "걸어 둔 캐시"가 한 번도 맞지 않았다

- **측정부터**: 운영 요청 로그 일주일치를 나누니 첫 화면의 약 30%가 콜드스타트(중간값 8.3초)였고, "로그인이 느리다"는 체감과 달리 로그인 자체는 1초 안쪽, 느린 것은 로그인 **직후** 첫 화면(7.3초)이었다.
- **원인 1 — 캐시 열쇠**: Next `fetch` 캐시는 요청 헤더까지 열쇠에 넣는다(`generateCacheKey` 소스로 확인). CRM 토큰의 발급 시각과 구글 API의 새 access token 때문에 열쇠가 매번 달라 캐시가 **한 번도 맞지 않았다.** 응답이 보는 사람과 무관하다는 것을 CRM 코드로 확인한 뒤 인스턴스 메모리 캐시로 교체.
- **원인 2 · 3**: 명부가 지사별 내선을 약 20명분(한 사람 1.5~4.9초) 다 받을 때까지 기다림 → 기억한 값으로 먼저 그리고 뒤에서 갱신. 구글 캘린더 대기 → 일정 칸만 따로 불러오고 사람별로 기억.
- **성과**: `GET /` **7.3초 → 4.48초 → 1.50초**(원인을 하나씩 걷어낼 때마다 재측정). 내선 통화 확인창은 순차 조회를 동시로 바꾸고 경유 서비스를 빼 2.2초 → 평소 1.36초. 콜드스타트는 최소 인스턴스 1(월 약 $14 추정)로 막고, 화면이 더 이상 기다리지 않게 된 연결 서비스는 0으로 되돌려 비용 증가를 한 대분으로 제한.

V. 법무법인 법승 공식 홈페이지(레거시) 유지보수

- **기간**: 2026-01 – 진행 중
- **기술스택**: ASP.NET Core 6.0 · CSHTML · Vanilla JavaScript/CSS · SQL Server · Windows Server/IIS · FileZilla
- **운영 환경**: 가비아 호스팅 Windows Server(IIS)에 직접 배포·운영. 도메인 등록과 DNS 존(홈페이지 · CRM · 사건관리 서브도메인 포함)도 가비아에서 관리.
- **담당 범위**: 기존 대고객 홈페이지(lawwin.co.kr)의 화면 개선, 검색엔진 대응, 백엔드 API 개발, 소스 이관 및 배포 환경 분석, 서버 장애 원인 분석, DNS 전환·롤백, 신규 홈페이지(V2)와의 데이터 동기화. 외주 소스를 사내 저장소로 이관하고 운영 절차를 문서화해 직접 유지보수할 수 있는 기반을 마련.

담당 업무 및 핵심 문제 해결 사례

1) 변호사 상세 페이지 진입 지연과 모바일 레이아웃 개선 — 2026-01

- **문제**: 변호사 상세 페이지 진입에 2~3초 지연이 발생하고, 모바일에서 검색창·소개 영역·이동 화살표가 겹치는 문제가 있었음.
- **조치와 결과**: Chrome DevTools로 `member.cshtml`의 불필요한 prefetch 스크립트와 이벤트 핸들러를 찾아 정리해 진입 지연을 해소. 절대 좌표에 의존하던 화면에는 Flex 레이아웃과 `max-width` 미디어 쿼리를 적용해 모바일 요소 간 간섭을 수정.

2) 검색용 URL 정리와 뒤로가기 상태 보존 — 2026-02

- **문제**: 상세 페이지 URL에 `ReturnUrl` 쿼리 파라미터가 노출되어 주소 정리가 필요했으며, 단순 제거 시 이전 목록으로 돌아가는 경로가 유실될 수 있었음.
- **조치와 결과**: URL에서 해당 파라미터를 제거하고 Web Storage에 복귀 경로를 저장·복원하도록 변경해 검색용 주소 정리와 기존 탐색 동작을 함께 유지.

3) 전국 10개 지점 사이트맵 갱신 자동화 — 2026-03

- **작업과 가치**: Playwright로 전국 10개 지점의 콘텐츠를 크롤링하고 XML 사이트맵을 갱신하는 자동화 스크립트를 구축. `robots.txt`에 지역별 사이트맵을 등록해 지점별 콘텐츠 갱신을 검색엔진에 전달할 수 있는 관리 경로를 정리.

4) 외주 ASP.NET 소스 이관과 유지보수 구조 문서화 — 2026-04

- **작업**: 외주 개발 ASP.NET Core 6.0 소스를 사내 저장소로 최초 이관하고, WEB/BLL/DAL 3계층의 역할과 빌드·업로드 배포 절차를 문서화.
- **결과**: 기존 시스템을 사내에서 분석·수정하고 배포 절차를 확인할 수 있는 유지보수 기반을 마련.

5) 외부 업체 SEO 데이터 업로드 API와 URL 매칭 개선 — 2026-04

- **작업**: `POST /api/seo/upload`와 `X-API-KEY` 인증을 구현하고, `wwwroot/seo` 디렉터리가 없을 때 자동 생성하도록 방어 처리.
- **고도화**: URL별 문구 조회에 쿼리스트링·후행 슬래시 정규화를 적용해 주소 표기 차이에도 일관되게 매칭되도록 개선. 업로드 데이터는 메모리에 캐싱하고, 빌드 시 `publish_output` 폴더가 중복 포함되지 않도록 산출물 규칙도 정리.

6) HTTP 500.30·500.31과 DB 연결 오류의 원인 분리 — 2026-04

- **문제**: 기존 홈페이지의 500 계열 접속 오류에 대해 애플리케이션 코드와 서버 환경 중 어느 계층을 조치해야 하는지 판단이 필요했음.

- **분석과 결과**: 500.30·500.31은 서버 디스크 부족에 따른 호스팅 번들(DLL) 설치 누락, 일반 500은 `appsettings.json`의 DB 주소 매핑 오류로 원인을 분리. 디스크 확보·번들 재설치와 DB 설정 수정 등 후속 조치의 범위를 구체화.
- 해당 기록에서는 원인 규명까지 확인되며, 서버 복구는 후속 과제로 남아 있음.

7) FileZilla 업로드와 IIS 오류 대응 절차 정리 — 2026-05

- **작업과 가치**: 반복되는 FileZilla 업로드 이슈와 DLL 버전 불일치 등 IIS 500.x 오류의 대처 방법을 README에 정리해, 배포 시 확인할 항목과 팀 내 트러블슈팅 기준을 문서화.

8) 가비아 DNS로 실서버 컷오버와 롤백 — 2026-08

- **컷오버 설계**: 실서버 전환을 "가비아를 닫는 일"이 아니라 **A 레코드 2줄 교체**로 정의한 런북을 작성. 도메인 등록 · DNS 존 · 레거시 IIS는 유지해야 했다 — 업로드 원본 파일이 아직 레거시 서버를 참조했고, 같은 존의 CRM · 사건관리 서브도메인 레코드는 건드리면 안 됐기 때문. 소유 확인 TXT와 인증서 갱신용 CNAME은 삭제 금지 항목으로 표시하고, 가비아 TTL 최소값이 600초라는 것을 확인해 롤백 소요 예상을 5분에서 10분으로 정정.
- **롤백 수행(8/11)**: 광고대행사의 SEO 노출 단절 보고와 경영 판단으로 `www`를 레거시로 되돌림. 넘기기 전에 레거시 서버를 먼저 실측(HTTPS 200, 응답 0.12~0.17초, 인증서 유효기간, robots · 지사별 사이트맵 10개)하고, 전환 후에는 공개 DNS 반영과 **레거시 IP 직접 호출 시 신규 사이트 흔적 0건**으로 정상 서빙을 확인. 클라우드 쪽 로드밸런서 · 인증서 자원은 지우지 않고 유지해, 신규 홈페이지를 새 도메인으로 옮길 때 그대로 재사용.

9) 롤백 후 신규 사이트에만 남은 상담 42건을 레거시 DB로 이관 — 2026-08

- **문제**: 컷오버 기간(8/6~8/11) 상담 51건이 신규 사이트 DB에만 남아 레거시 관리자 화면에서 보이지 않았고, 담당 변호사 배정이 누락될 위험이 있었음.
- **걸린 것 세 가지**: ① 레거시는 한국시간을 시간대 없이 저장하고 이관본은 그 숫자에 UTC 라벨만 붙어 있어, 기존 행을 대조해 확인한 뒤 **+9시간 보정** ② 컷오버 당일 양쪽이 같은 번호를 각자 발급해 **번호가 충돌** → 원본 번호 강제 삽입(`IDENTITY_INSERT` + RESEED)은 운영 DB에서 위험해 레거시가 새로 부여하게 함(조회는 접수 고유키 기반이라 영향 없음) ③ 레거시 IP 칸이 `varchar(20)`인데 모바일 IPv6는 최대 39자이고 **SQL Server는 초과분을 자르지 않고 오류를 냄** → 해당 11건은 NULL로 넣고 원본은 주석에 보존.
- **결과**: 테스트 9건을 제외한 **42건 삽입, 중복 0건**.

10) 레거시 → 신규 홈페이지 데이터 동기화 운영 — 2026-09

- **2주 침묵의 진짜 원인**: 동기화가 멈춘 원인을 "레거시 MSSQL 1433 포트 차단"으로 기록했으나, 재측정해 보니 레거시가 실제로 쓰는 포트는 `appsettings.json`이 가리키는 **49752**였고 열려 있었음. 1433은 원래 쓰지 않는 포트라 막힌 게 정상이었고, 끊긴 곳은 클라우드 DB 허용목록 한 곳이었다 — 레거시 방화벽은 손댈 필요가 없었다.
- **구조 전환**: 양쪽 DB에 직접 붙던 구조를, 레거시 DB를 읽어 **신규 백엔드 HTTPS API로 밀어 넣는 방식**으로 바꿔 인바운드 개방 · 허용목록 의존을 없앰. 고정 IP용 NAT 안(연 약 60만 원)과 .NET 운영 서버에 런타임을 얹는 안은 기각. 접속 정보는 레거시 웹앱 `appsettings.json`을 직접 읽어 비밀번호 이중관리를 없앴다(끝에 쉼표가 남은 비표준 JSON이라 정규식으로 추출).
- **예약 발행 글 38건**: 레거시는 예약 글을 비공개 + 미래 작성일로 넣었다가 당일 공개로 바꾸는데, 배치가 기존 행을 다시 보지 않아 신규 사이트에서 영구히 숨겨져 있었음 → 매 회차 최근 60일 글의 공개 여부와 작성일을 레거시에 맞추게 하고, 한국시간을 UTC로 읽던 9시간 오차도 함께 수정.
- **결과**: 게시물 61건 · 상담 305건 · 예약 글 38건 복구, 매일 자동 실행 + 실패 즉시 알림 + 26시간 침묵 감시.

11) 레거시 업로드 원본 의존 정리와 운영 대응 — 2026-06 ~ 10

- **이미지 서빙 전환(6월)**: 신규 사이트가 이미지 요청을 가비아 레거시 서버로 프록시하던 구조를 GCS 서빙으로 바꾸고, 그 과정에서 공지 · 사회공헌 이미지가 안 뜨던 경로 매칭 버그를 수정.
- **새로 올라온 파일 누락(9월)**: 게시물은 동기화되지만 레거시에 새로 올린 파일은 GCS에 없어 판결문 이미지 등이 404였음. GCS에 없으면 레거시 원본 서버로 보내는 폴백을 썸네일에서 모든 업로드 경로와 대문자 `/Uploads`까지 넓히고, 누락 본문 사진 **97장을 수동 복구해 재대조 누락 0**을 확인. 레거시 IIS를 끌 수 없는 이유가 바로 이 원본 의존이라, 파일 이관을 동기화에 넣는 것을 후속 과제로 남김.
- **운영 공지**: 연휴 상담 안내처럼 두 사이트에 동시에 걸어야 하는 공지는, 레거시용으로 기존 팝업 형식에 한국시간 판정을 넣은 CSHTML 블록을 만들어 반영 — 끄는 날 재배포가 필요 없다.

VI. GCP 인프라 / DevOps — 여러 시스템 공통 적용 원칙

- **배포 안전 원칙 전파**: 사건관리 앱에서 발생한 `--source=.` 오배포 장애를 계기로 "빌드 · 배포 2단계 분리 + 배포 후 응답 본문 검증"을 원칙화하고, 이후 신규 시스템(사내 포털)에도 선제 적용.
- **Cloud Scheduler 기반 예열/해제 스케줄링**: 서비스 레벨 `--min-instances` + Cloud Scheduler로 업무시간(08:30–19:30 KST)만 인스턴스를 예열하고, 리비전 재배포 없이 토글되도록 설계(사건관리 앱). 사내 포털은 실측(첫 화면의 30%가 콜드스타트)을 근거로 상시 1대를 택하는 등, 서비스마다 측정값으로 정한다.
- **실패는 조용히 덮지 않는다(fail-closed)**: 폴백이 실패를 가려 몇 주~몇 달간 정상으로 보이던 사례(동기화 배치 종료코드 0, 권한 연동 폴백, 빈 편집자 명단)를 계기로, 권한 판정은 원본을 못 읽으면 막고, 배치는 실패 알림과 별도로 "일정 시간 실행 기록 없음"을 감시하는 구조를 기본값으로 둔다.
- **캐시는 "걸었다"가 아니라 "맞는다"를 측정**: 캐시 열쇠에 인증 헤더가 섞여 한 번도 적중하지 않던 사례 이후, 캐시를 둘 때 무엇을 · 어디에 · 얼마나 기억하는지 표로 문서화하고 적중 여부를 응답 시간으로 확인.
- **시크릿 관리 원칙**: 묶음 `.env` 대신 전용 Secret Manager 시크릿으로 분리해 "빠뜨리면 배포 자체가 실패"하는 구조로 전환.
- **비용 우선 실측 원칙**: 리전별 Cloud Run 요금 티어(서울 = Tier 2), egress 단가 등을 "주석에 적힌 값"이 아니라 공식 문서 · 실측으로 재확인해 웹소켓/SSE/폴링 등 기술 선택에 직접 반영.

IT 솔루션 기업 (개발팀)

인턴(2024-08–2025-02) → 사원(2025-03–2025-06). 일학습병행제 기반으로 실무 프로젝트 및 유지보수 업무를 수행하며, 월 단위 평가로 실무 이해도와 기술 숙련도를 검증받음.

VII. 공공 연구기관 — CCTV 실시간 모니터링 웹서비스 유지보수

- **기간**: 2025-03 – 2025-06 (사원)
- **기술스택**: Node.js, FFmpeg, Linux(Ubuntu)
- **서비스/용도**: 공사 현장의 CCTV 스트림을 웹 UI에서 실시간으로 모니터링하는 관리 시스템.

핵심 업무

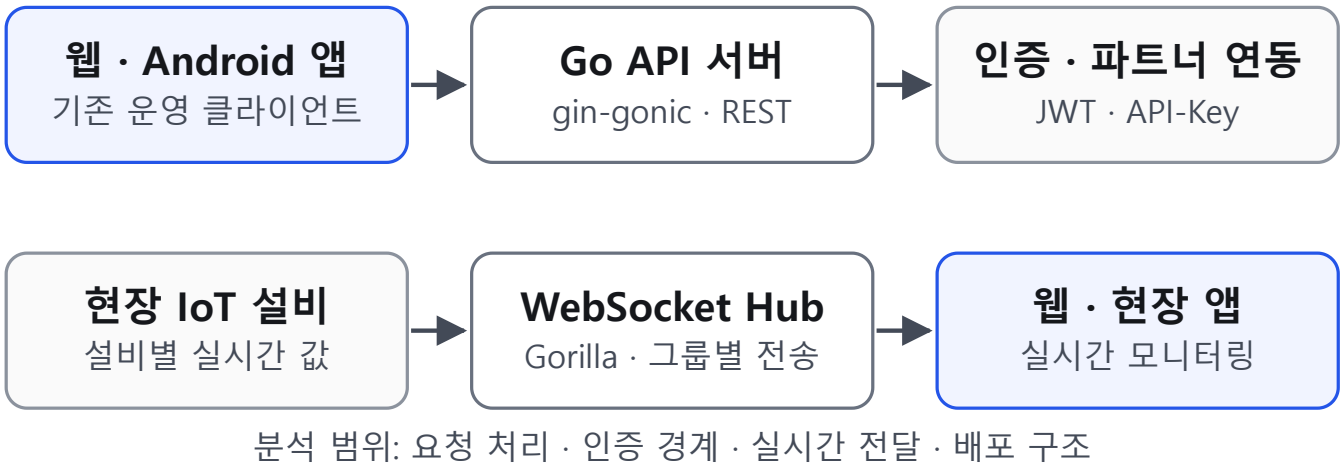
- Node.js 기반 CCTV 실시간 모니터링 웹 서비스 유지보수.
- FFmpeg 스트림 처리 로직을 Windows / Ubuntu 환경별로 분기 적용.
- Ubuntu 환경에서 FFmpeg를 리눅스 스크립트 형태로 실행 · 배포하여 운영 환경 대응.

VIII. 에너지 공기업 — 신규 개발 착수를 위한 기존 시스템 코드 분석

- **기간**: 2025-03 – 2025-06 (사원)
- **직무 성격**: 가스 누수 의심 등 고객 이벤트 요청 → 직원 점검 → 운영자 현황 확인으로 이어지는 앱의 **신규 개발에 앞서, 기존에 운영 중이던 시스템의 코드 흐름 전체를 분석**하는 역할.

아래 아키텍처와 기술 상세는 **본인이 구현한 것이 아니라**, 신규 개발 설계 근거자료를 만들기 위해 기존 운영 시스템(Go 백엔드 + JavaScript 프론트엔드)을 직접 읽고 분석해 파악한 내용입니다.

분석 대상 시스템 구조



분석한 배포 구조: Docker 멀티스테이지 빌드로 생성한 Go 바이너리를 Linux systemd 서비스로 운영. 아래 항목은 기존 시스템을 읽고 파악한 내용이며 본인의 신규 구현 범위와 구분합니다.

분석을 통해 파악한 내용

- **백엔드 로직 이해**: Go(gin-gonic) 기반 REST API 서버에서 가스 점검 이벤트(Order/Task/Target) 생성 · 조회 · 수정 로직과, 점검 프로세스 유형(4종)마다 "처리 시작 → 상태 갱신 → 재오픈 → 목표값 저장"이 반복되는 API 패턴을 파악.
- **프론트엔드 구조 이해**: JavaScript(CommonJS) 기반 프론트엔드(서버렌더링 템플릿 + vanilla JS)의 코드 구조를 파악해 점검 요청 · 응답 UI 상호작용 로직을 이해.
- **연동 관계 분석**: 프론트엔드 ↔ 백엔드 REST API 연동 관계를 분석하고 API 요청/응답 흐름을 검증.
- **인증 체계 확인**: 사용자 JWT(쿠키 기반) + 파트너사 API-Key + 특정 민감 엔드포인트 대상 토큰 화이트리스트로, 신뢰 경계가 3단으로 분리된 인증 구조임을 확인.
- **실시간 처리 구조 확인**: Gorilla WebSocket 기반 허브가 설비(IOT) 단위로 클라이언트를 그룹핑해 데이터를 브로드캐스트하는 구조, 논블로킹 전송(`select` / `default`) 처리 방식을 코드 레벨로 확인.
- **배포 구조 확인**: Docker 멀티스테이지 빌드(정적 바이너리) + Linux systemd 서비스로 운영되는 배포 방식을 확인.
- **문서화**: 디렉터리 구조와 컴포넌트 단위 코드 흐름을 분석해 유지보수성과 전체 코드 구조에 대한 이해를 신규 개발팀에 전달할 수 있도록 정리.

IX. 제조업 고객사 — 공장 센서데이터 웹 유지보수 · UI 개선

- **기간**: 2024-10 – 2025-02 (인턴)
- **기술스택**: Spring Framework, PostgreSQL, JavaScript, CSS
- **서비스/용도**: 공장 현장 데이터(온도 · 습도 · 풍향 등)를 시각화 · 관리하는 B2B 운영 웹 시스템.

핵심 업무

- PostgreSQL 센서 데이터 기반으로 Spring Framework 서비스의 데이터 출력 · 가공 로직 수정 및 화면 연동 구조 개선.
- JavaScript/CSS로 화면 이벤트 · UI 기능 개선, 레거시 코드 리팩토링으로 유지보수성 · UI 안정성 향상.